A Reference Checker for Input Specifications

Formal contract, implementation, and worked example

Research author: Dragan Stosic, with Codex assistance in implementation and documentation.
Date: 5 November 2026.
Project: `alsomitra-fixes/contribution/spec_compiler`

The reference checker determines whether a given input satisfies a logical specification. It preserves the meaning of that specification across a change of coordinates: a condition expressed in physical quantities and a check performed on a normalized input must give the same answer. This agreement is proved in Lean for the entire supported language.

The implementation is a standalone prototype within the project. It provides an executable compiler and checker over exact rational numbers, which can serve as a reference when evaluating another implementation of specification translation. Integration of this general language into the Python/PyTorch workflow remains to be implemented.

1. Inputs and result

The checker takes a physical specification, a normalization, and a normalized point. It returns `true` if the point, interpreted in physical coordinates, satisfies the specification; otherwise it returns `false`.

Item	Meaning	Example
Specification	A condition on physical coordinates	`10 <= x <= 12 OR 18 < x <= 20`
Normalization	An offset and a positive scale for each coordinate	`z = (x - 10) / 10`
Point	Values expressed in normalized coordinates	`z = 9/10`
Result	Membership in the specified input region	`true`, because physical `x = 19`

A `true` result establishes membership in the input region. The neural network's output is not an argument to this function. Establishing that the network produces a safe output on such an input is a separate property.

The term *reference* means that the checker has a precise semantics and a proof connecting its decisions to that semantics. When comparing it with another program, both must receive the same specification and the same numerical interpretation of the input.

2. Language and mathematical objects

An input is a function assigning a number to each coordinate. The coordinate type may represent a single quantity, the six Alsomitra coordinates, or another index set. Each formula has a finite structure.

The language supports comparisons between a coordinate and a constant:

```
x[i] <= k     x[i] < k     x[i] >= k     x[i] > k

TRUE          FALSE       AND          OR          NOT
```

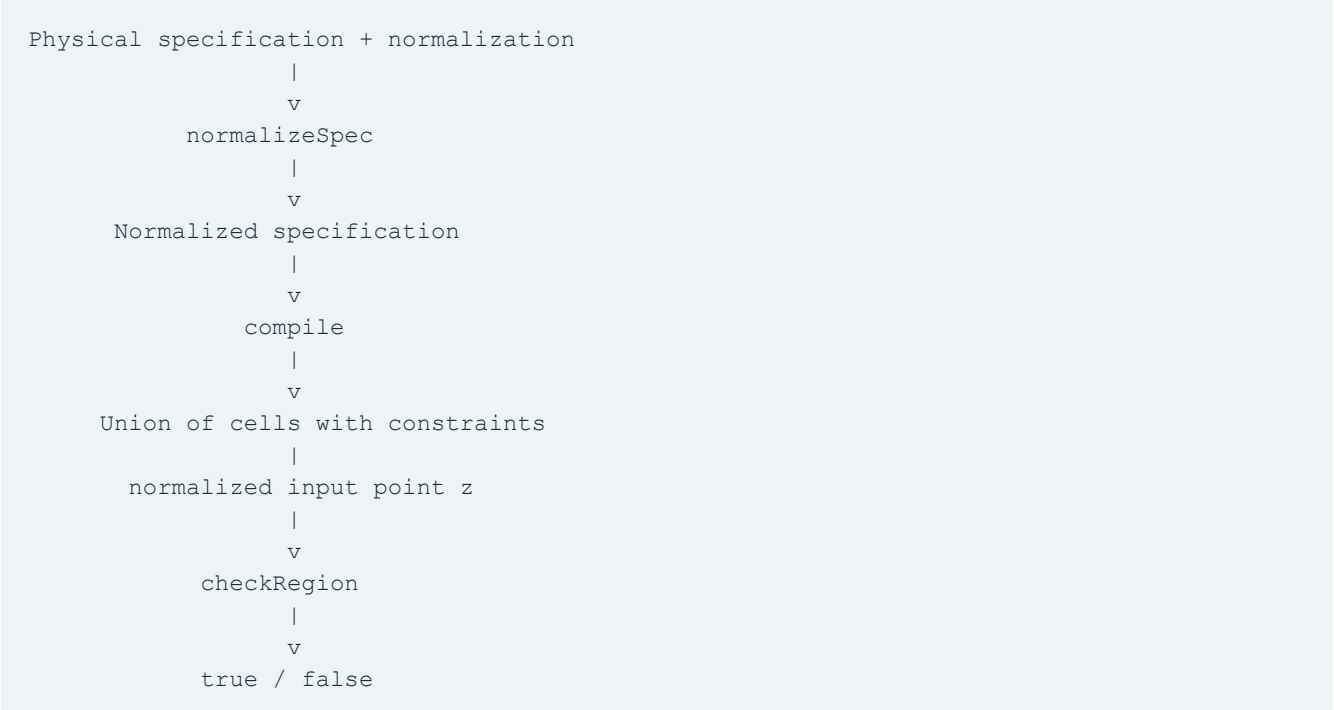
This is a deliberately restricted language. An expression such as `x + y <= 2`, involving the sum of two variable coordinates, is not directly supported as an atom. In the Alsomitra example, `anchorX + y <= 2` can be rewritten as `y <= 2 - anchorX`, because `anchorX` is a fixed specification parameter.

Code object	Role
`Atom`	One comparison: a coordinate, a comparison operator, and a threshold.
`Formula`	A logical syntax tree built from atoms and connectives.
`Cell`	A list of atoms that must all hold.
`Region`	A list of cells; satisfying any one cell is sufficient.
`PhysicalSpec`	A formula whose thresholds use physical coordinates.
`NormalizedSpec`	A formula whose thresholds have been transformed into normalized coordinates.
`AffineCoordinates`	Offsets, scales, and a proof that every scale is positive.

A cell describes an intersection of axis-aligned constraints. It may be unbounded, empty, or contain redundant constraints. A region represents a union of these cells. The current implementation retains lists of constraints; it does not automatically reduce them to canonical lower and upper bounds.

3. From specification to decision

The function `checkNormalized` first normalizes the thresholds, then compiles the formula into a region, and finally checks whether the point belongs to that region.



Semantically, the same answer can be obtained by a different route: denormalize `z` into the physical point `x` and interpret the original formula directly. The main theorem states that these two routes agree for every admissible choice of arguments.

```
checkNormalized(n, s, z) = true
              IF AND ONLY IF
s.formula holds at denormalizePoint(n, z).
```

Denormalization in this theorem explains the meaning of the decision. The executable checker itself compares the normalized input against the transformed thresholds.

4. Worked example with two disconnected regions

Consider a scalar quantity `x`. We want to accept values from 10 to 12, including both endpoints, or values strictly greater than 18 and at most 20.

```
Phi(x) = (10 <= x AND x <= 12)
         OR
         (NOT (x <= 18) AND x <= 20)

Region = [10,12] UNION (18,20]
```

The round bracket at 18 means that 18 is excluded. Values in the gap between 12 and 18 are also excluded. This is a deliberately constructed example illustrating the semantics, rather than an additional defect found in the Alsomitra code.

4.1. Transforming the thresholds

Choose an offset of 10 and a scale of 10:

```
z = (x - 10) / 10
x = 10*z + 10
```

Each physical threshold `k` becomes `(k - 10) / 10`. Because the scale is positive, the direction of each inequality is preserved. Strict inequalities remain strict.

Physical condition	Normalized condition
`x >= 10`	`z >= 0`
`x <= 12`	`z <= 1/5`
`NOT (x <= 18)`	`NOT (z <= 4/5)`, equivalently `z > 4/5`
`x <= 20`	`z <= 1`

The resulting region is exactly:

```
[0,1/5] UNION (4/5,1]
```

4.2. The compiled cells

The compiler produces two cells. Both constraints must hold within a cell; the results for the two cells are then combined using `OR`.

```
Region = [
  Cell 1: [z >= 0,   z <= 1/5],
  Cell 2: [z > 4/5,  z <= 1]
]

checkRegion = some cell has all its constraints satisfied.
```

The negation `NOT (z <= 4/5)` becomes `z > 4/5`. Translating it to `z >= 4/5` would change its meaning precisely at the boundary.

4.3. Results for concrete inputs

Physical value x	Normalized value z	Result	Explanation
10	`0`	`true`	The first region includes its left endpoint.
11	`1/10`	`true`	The point is inside the first region.
12	`1/5`	`true`	The first region includes its right endpoint.
15	`1/2`	`false`	The point lies in the gap between the regions.
18	`4/5`	`false`	The second region excludes its left endpoint.
19	`9/10`	`true`	The point is inside the second region.
20	`1`	`true`	The second region includes its right endpoint.

For `z = 4/5`, the first cell fails because `4/5 <= 1/5` is false. The second fails because `4/5 > 4/5` is false. The result is `false`. Denormalization gives `x = 18`, so the physical specification yields the same decision.

For `z = 9/10`, the first cell again fails. In the second cell, both `9/10 > 4/5` and `9/10 <= 1` hold, so the result is `true`. Denormalization gives the permitted value `x = 19`.

4.4. The exact meaning of numerical values

In this example, `4/5`, or the decimal notation `0.8`, denotes an exact rational number. The checker executes using `Rat`, rather than rounded float32 arithmetic.

The float32 value obtained by rounding decimal `0.8` is exactly `13421773/16777216`, which is slightly greater than `4/5`. If a future adapter passes that stored value without changing its meaning, the checker will accept it in the second region. This agrees with the specification: the input is slightly greater than the excluded boundary, rather than equal to it.

A reference comparison therefore requires agreement on whether the input means an intended decimal value or the exact value actually stored in memory. A general Python adapter performing that conversion is not part of this prototype.

5. What the correctness proof establishes

The theorem `checkNormalized_correct` establishes both directions of the equivalence.

Soundness of acceptance: if the checker returns `true`, the denormalized physical point satisfies the source formula. Compilation and normalization have not introduced an inadmissible point.

Completeness of membership checking: if the denormalized physical point satisfies the source formula, checking the corresponding normalized point returns `true`. Compilation has not lost an allowed component or endpoint. This completeness concerns the decision for a supplied point; it is not a claim about an algorithm that finds all such points.

The proof is not inferred from the seven values in the table. It applies to every formula in the language, every choice of thresholds, every positive affine normalization, and every point. The compilation theorem requires a linear order on scalars. The normalization theorems require an ordered field and therefore cover mathematical real numbers and rational numbers. The concrete execution here uses rational numbers.

The proof is compositional:

Theorem	What it connects
`flip_correct`	Negation of an atomic comparison and its exact complement.
`compile_correct`	Formula semantics and membership in the compiled region.
`normalized_formula`	The normalized formula and the physical formula evaluated at the denormalized point.
`normalization_compilation_commute`	The semantics of compiling before and after normalization.
`checkRegion_correct`	The executable Boolean result and mathematical region membership.
`checkNormalized_correct`	The complete checking procedure and the original physical specification.

The functions covered by these proofs are the same Lean definitions executed in the rational demonstration. Execution additionally depends on the Lean compiler, runtime, and environment; those components are outside the scope of these new theorems.

6. Negative controls that expose changes in meaning

`RegionExamples.lean` contains deliberately modified variants of the procedure and proofs that they give a different answer on a concrete witness.

Change	Witness	Correct result	Modified result
Replace the union with its enclosing interval `[10,20]`	`x = 15`	`false`	`true`
Include the excluded endpoint 18	`x = 18`	`false`	`true`
Drop the second component	`x = 19`	`true`	`false`
Normalize thresholds that are already normalized	`x = 11, z = 1/10`	`true`	`false`

The distinct types for physical and normalized specifications prevent a direct call to `normalizeSpec` on its own normalized output. Explicitly rewrapping the underlying formula in the wrong type bypasses that protection; the double-normalization counterexample deliberately does so.

A separate theorem shows that the inconsistent condition `x >= 2 AND x <= 1` rejects every point. The compiler may retain this empty cell in the region representation, but the checker will never accept it.

These cases demonstrate the reference procedure's sensitivity to important changes in semantics. They do not assert that all these defects occur in a particular external library.

7. Connection to Alsomitra Property2

An Alsomitra input has six coordinates: `vx`, `vy`, `omega`, `theta`, `x`, and `y`. In the existing project, the Property2 precondition has the form:

```
p belongs to the physical domain D
AND 0 <= anchorX + p.y
AND anchorX + p.y <= bandUpper
AND p.vy <= velocityUpper
AND p.omega <= angularUpper
```

For `anchorX = 10` and the default thresholds, this gives `-10 <= p.y <= -8`, `p.vy <= -3/10`, and `p.omega <= -3/25`, together with the domain constraints in `D` on all six coordinates. The domain retains the same six-decimal constants as the earlier project definition.

`anchorX` is a fixed parameter. It is not replaced with the candidate's variable coordinate `p.x`. This preserves direct agreement with the existing contract relative to a fixed anchor.

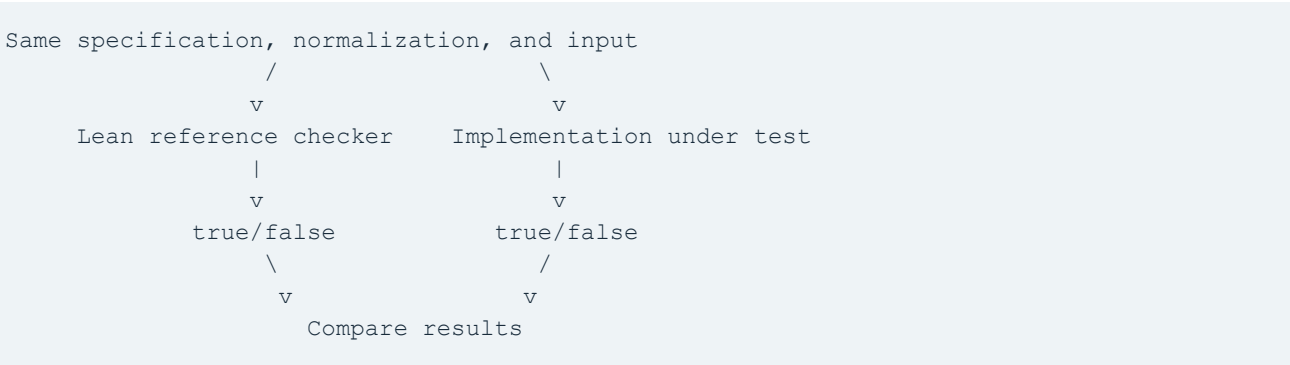
`RegionAlsomitra.lean` expresses this condition in the general language and proves:

```
the general checker accepts z
          IF AND ONLY IF
the existing Property2 precondition holds at denormalized z.
```

The theorem covers every rational choice of parameters and every positive normalizer. It is not restricted to the default thresholds or a selected network. Agreement with the earlier specialized compiler for the rectangular region is also proved.

8. Using the checker as a reference for another program

To compare the checker with a Python implementation, first fix the same formula, normalization, and input values. Then compare that program's decision with the reference checker's decision.



A disagreement provides a concrete case to investigate: for example, an incorrectly translated boundary, incorrect normalization, a lost component, or a different numerical interpretation. A disagreement alone does not identify which of these caused it.

The prototype already executes the Lean code and compares its results with an independent Python evaluator using `Fraction`. That evaluator interprets the source formula directly at the physical point. It does not repeat the compilation algorithm that constructs cells.

Use within the existing PyTorch workflow requires an adapter that transfers specifications, normalizations, and points into an agreed exact representation. This is a further integration step. The function `acceptCandidate`

already provides a proved acceptance decision for a supplied candidate; it does not include a general point generator or projection algorithm.

9. Relationship to the loss function

A separate positive fragment, consisting of non-strict comparisons combined with `AND` and `OR`, has a nonnegative loss function:

```
loss(x <= k) = max(0, x-k)
loss(x >= k) = max(0, k-x)
loss(P AND Q) = loss(P) + loss(Q)
loss(P OR Q)  = loss(P) * loss(Q)
```

For this fragment, the loss is proved to be zero if and only if the formula holds. Agreement between normalized loss and the checker is also proved. This concerns the meaning of zero loss at a supplied point; training success and optimizer behavior are separate questions.

For the strict condition `x < 0`, the same promise cannot be met by a continuous real-valued loss function. If a continuous function is zero at every negative value, continuity forces it to be zero at 0 as well. But `0 < 0` is false. The theorem `strict_boundary_obstruction` formalizes this standard topological obstruction.

The general checker therefore handles strict boundaries exactly, while the theorem about a continuous loss with exact zeros applies to the separately defined fragment. This restriction concerns that particular loss criterion, rather than every possible numerical interpretation of logic.

10. Reproduction and verification status

10.1. Prepare a fresh checkout

Use a checkout containing `contribution/spec_compiler`. The commands below assume a macOS or Linux shell, Python 3.11 or newer with virtual-environment support, Git, and an installed Lean toolchain manager (`elan`) with `lake` on `PATH`. Run them from the repository root: the directory containing `README.md`, `lean/`, and `contribution/`. No path specific to the author's machine is required.

The repository selects Lean 4.35.0 through `lean/lean-toolchain` and pins Mathlib through `lean/lake-manifest.json`. Prepare its dependencies and a Python environment:

```
# Start in the alsomitra-fixes repository root.
cd lean
lake exe cache get
lake env lean --version
cd ..

python3 -m venv contribution/.build/checker-env
. contribution/.build/checker-env/bin/activate
python -m pip install torch
```

The version command should report Lean 4.35.0. The initial setup downloads the required dependencies. The Python environment is inside the ignored build directory.

The current verification runner checks the source and compiled artifact for `Property2` against the receipt produced by `verify_float32.py`. Therefore, on a fresh checkout, first run:

```
python -B contribution/verify_float32.py
```

This builds the shared rational definitions, runs their checks, and produces the float32 verification receipt. It requires PyTorch, but does not require an upstream checkout, trained weights, or a training run. The specification compiler's mathematical theorems remain independent of PyTorch. Wait for this command to finish successfully before proceeding.

10.2. Compile and verify the reference checker

With the Python environment still active and the shell at the repository root, run:

```
python3 -B contribution/spec_compiler/verify.py
```

The script rebuilds the six Lean modules, audits the theorem dependencies, and compares the executable checker with the independent rational evaluator. Its final success line is:

```
PASS: 42 audited roots; 21931 runtime comparisons
```

It writes the detailed report to `contribution/results/region_compiler_verification.json`. Look for `"status": "PASS"`. A missing `lake` command means the toolchain is not on `PATH`; a missing `torch` module means the Python environment is not active or its installation is incomplete. A missing `.build/Property2.olean` or a stale `Property2` receipt means the prerequisite command in Section 10.1 must be rerun.

The latest recorded verification, dated 5 November 2026, has status `PASS`. It checks and audits 42 theorem roots in Lean. The audit permits only the standard axioms `propext`, `Classical.choice`, and `Quot.sound`; it rejects `sorry` and unchecked native-decision axioms.

The executable comparison covers 812 formulas under three normalizers at nine points, plus the seven values in the worked example:

```
812 * 3 * 9 + 7 = 21,931 comparisons
```

These finite executions supplement the universal theorems. They do not establish correctness on untested inputs; the Lean theorems provide that justification. The report records hashes of the sources and compiled artifacts, binding the successful result to the checked files.

10.3. Run the small example

A minimal example using the checked definitions is provided in `Example.lean` alongside this guide:

```
import RegionExamples

open Contribution.RegionCompiler
open Contribution.RegionCompiler.Examples

#eval checkNormalized units disconnected (point (4/5))
-- false: physical x = 18, an excluded boundary

#eval checkNormalized units disconnected (point (9/10))
-- true: physical x = 19

#eval checkNormalized units disconnected
  (point (13421773/16777216))
-- true: the exact float32 value of 0.8 is greater than 4/5
```

To run this short example, start at the repository root:

```
cd lean
LEAN_PATH=../contribution/.build lake env lean \
  ../contribution/spec_compiler/docs/Example.lean
cd ..
```

The verification in Section 10.2 must have completed first. The expected output, in order, is:

```
false
true
true
```

These are the decisions for the exact boundary `4/5`, the interior point `9/10`, and the exact rational value of float32 `0.8`. To try a different point, edit a `point (...)` argument in `Example.lean` and rerun the example command. To try another specification, add a separate example using the constructors illustrated in `RegionExamples.lean`; retain the existing regression examples and their expected results. The final `cd ..` returns the shell to the repository root.

11. Source map and scope

File in `contribution/spec_compiler`	Contents
`RegionSyntax.lean`	Language, cells, regions, and compilation proof.
`RegionNormalization.lean`	Coordinate types, normalization, and semantic preservation.
`RegionCheck.lean`	Executable checker, candidate acceptance, and contract transport.
`RegionAlsomitra.lean`	Connection to the existing Property2.
`RegionLoss.lean`	Supported loss fragment and the strict-boundary obstruction.
`RegionExamples.lean`	Worked example and deliberate changes in semantics.
`RegionMain.lean`	Executable examples for comparison.
`verify.py`	Build, proof audit, and independent execution checks.

The full verification result is in `contribution/results/region_compiler_verification.json`. The scope and research assessment are described in `contribution/spec_compiler/RESEARCH_ASSESSMENT.md`.

This addition provides a precise, verified reference procedure for the defined specification fragment. It does not prove a property of a particular neural network, verify arbitrary Python programs, or provide a general efficient point generator. Compilation into disjunctive normal form may produce exponentially many cells.

Relevant context is provided by the Alsomitra paper, Vehicle, and LDL. Preserving meaning across specifications, loss functions, and verification has substantial precedents. The current result is a verified prototype; its independent research novelty has not been established. The detailed comparison is in `RESEARCH_ASSESSMENT.md`.